

PAUTA

Guía de Ejercicios: Funciones en Python

Introducción a la Programación

NIVEL BÁSICO — EJERCICIOS 1, 2 Y 3

Ejercicio 1 Saludar con nombre

★ Básico

a) Pseudocódigo modelo

```
FUNCION saludar(nombre)
    INICIO
        ESCRIBIR "¡Hola, " + nombre + "! Bienvenido/a a programación."
    FIN
FIN FUNCION

// Programa principal
saludar("Ana")
saludar("Luis")
saludar("María")
```

b) Código Python modelo

```
def saludar(nombre):
    print(f"¡Hola, {nombre}! Bienvenido/a a programación.")

# Llamadas
saludar("Ana")
saludar("Luis")
saludar("María")
```

Rúbrica de corrección

Criterio	Pts	Descripción
Pseudocódigo: función correctamente definida con parámetro	2 pts	Completo / 1pt parcial / 0 ausente
Pseudocódigo: llamada a la función (3 veces)	1 pt	1pt o 0
Python: uso correcto de def y parámetro nombre	3 pts	3/2/1/0
Python: print con f-string o concatenación correcta	2 pts	2/1/0

Python: 3 llamadas con nombres distintos

2 pts

2/1/0



Aceptar concatenación con + además de f-strings.
 Aceptar comillas simples o dobles indistintamente.
 Si el mensaje está levemente diferente pero tiene nombre + mensaje de bienvenida: puntaje completo.

Ejercicio 2 Calcular el cuadrado

★ Básico

Puntaje: 10 pts

a) Pseudocódigo modelo

```

FUNCION cuadrado(n)
  INICIO
    resultado <- n * n
  RETORNAR resultado
  FIN
FIN FUNCION

// Programa principal
PARA i DESDE 1 HASTA 5 HACER
  ESCRIBIR i + "² = " + cuadrado(i)
FIN PARA

```

b) Código Python modelo

```

def cuadrado(n):
    return n ** 2      # También válido: n * n

for i in range(1, 6):
    print(f"{i}\u00b2 = {cuadrado(i)}")

```

Rúbrica de corrección

Criterio	Pts	Descripción
Pseudocódigo: función con return correcto	2 pts	2/1/0
Pseudocódigo: ciclo para llamar la función	1 pt	1/0
Python: función retorna n**2 o n*n	3 pts	3/2/0
Python: ciclo range(1, 6) correcto	2 pts	2/1/0
Python: salida con formato correcto	2 pts	2/1/0



Aceptar n**2 y n*n como equivalentes.

Aceptar range(1,6), range(1,5+1) o similar.

Si no usa el símbolo ² pero muestra el resultado correcto: descontar solo 1pt de formato.

Ejercicio 3 Mayor de dos números

★ Básico

Puntaje: **10 pts**

a) Pseudocódigo modelo

```

FUNCION mayor(a, b)
    INICIO
        SI a >= b ENTONCES
            RETORNAR a
        SINO
            RETORNAR b
        FIN SI
    FIN
FIN FUNCION

ESCRIBIR "El mayor entre 5 y 3 es: " + mayor(5, 3)
ESCRIBIR "El mayor entre 7 y 9 es: " + mayor(7, 9)
ESCRIBIR "El mayor entre 4 y 4 es: " + mayor(4, 4)
ESCRIBIR "El mayor entre -2 y 0 es: " + mayor(-2, 0)

```

b) Código Python modelo

```

def mayor(a, b):
    if a >= b:
        return a
    else:
        return b

# Alternativa válida con operador ternario:
# def mayor(a, b): return a if a >= b else b

print(f"El mayor entre 5 y 3 es: {mayor(5, 3)}")
print(f"El mayor entre 7 y 9 es: {mayor(7, 9)}")
print(f"El mayor entre 4 y 4 es: {mayor(4, 4)}")
print(f"El mayor entre -2 y 0 es: {mayor(-2, 0)}")

```

Rúbrica de corrección

Criterio	Pts	Descripción
Pseudocódigo: estructura if/else correcta con return	2 pts	2/1/0
Pseudocódigo: maneja el caso de igualdad	1 pt	1/0
Python: función con if/else y return correcto	4 pts	4/3/2/1/0
Python: al menos 4 llamadas con casos distintos	2 pts	2/1/0
Python: maneja correctamente los negativos	1 pt	1/0



Aceptar if a > b: return a / return b (caso igualdad también retorna b, que es correcto).
Aceptar el operador ternario de Python.
No aceptar uso de max() para esta función (el objetivo es programar la lógica).

NIVEL INTERMEDIO — EJERCICIOS 4 Y 5

Ejercicio 4 Calculadora de notas

★★ Intermedio

Puntaje: 15 pts

a) Pseudocódigo — promedio

```
FUNCION promedio(n1, n2, n3)
  INICIO
    total <- n1 + n2 + n3
    prom  <- total / 3
    RETORNAR redondear(prom, 2)
  FIN
FIN FUNCION
```

b) Pseudocódigo — estado

```
FUNCION estado(prom)
  INICIO
    SI prom >= 70 ENTONCES
      RETORNAR "Aprobado"
    SINO SI prom >= 50 ENTONCES
      RETORNAR "Recuperación"
    SINO
      RETORNAR "Reprobado"
    FIN SI
  FIN
FIN FUNCION
```

c) Código Python modelo

```
def promedio(n1, n2, n3):
    return round((n1 + n2 + n3) / 3, 2)

def estado(prom):
    if prom >= 70:
        return "Aprobado"
    elif prom >= 50:
        return "Recuperación"
    else:
        return "Reprobado"

# Programa principal
datos = [(75, 80, 90), (55, 60, 56), (30, 45, 53)]
```

```
for i, (n1, n2, n3) in enumerate(datos, 1):
    p = promedio(n1, n2, n3)
    print(f"Alumno {i}: promedio = {p} → {estado(p)}")
```

Rúbrica de corrección

Criterio	Pts	Descripción
Pseudocódigo promedio: fórmula y return correctos	2 pts	2/1/0
Pseudocódigo estado: if/elif/else con umbrales correctos	2 pts	2/1/0
Python promedio: round() y fórmula correcta	4 pts	4/3/2/0
Python estado: if/elif/else con los tres estados	4 pts	4/3/2/1/0
Python: llama ambas funciones para al menos 3 alumnos	3 pts	3/2/1/0



El umbral puede variar si el enunciado lo permite; lo importante es la estructura if/elif/else. Aceptar que llame promedio() dentro de estado() o que el programa principal maneje el flujo. Si usan 60 como umbral en vez de 70, revisar consistencia con el enunciado.

Ejercicio 5 Conversor de temperatura

★★ Intermedio

Puntaje: 15 pts

a) Pseudocódigo de las tres funciones

```
FUNCION celsius_a_fahrenheit(c)
    RETORNAR redondear((c * 9/5) + 32, 2)
FIN FUNCION

FUNCION fahrenheit_a_celsius(f)
    RETORNAR redondear((f - 32) * 5/9, 2)
FIN FUNCION

FUNCION celsius_a_kelvin(c)
    RETORNAR redondear(c + 273.15, 2)
FIN FUNCION
```

b) Código Python modelo

```
def celsius_a_fahrenheit(c):
    return round((c * 9/5) + 32, 2)

def fahrenheit_a_celsius(f):
    return round((f - 32) * 5/9, 2)

def celsius_a_kelvin(c):
    return round(c + 273.15, 2)
```

```
# Programa principal
temps = [0, 100, -40]
for t in temps:
    f = celsius_a_fahrenheit(t)
    k = celsius_a_kelvin(t)
    print(f"{t}°C → {f}°F / {k} K")
```

Rúbrica de corrección

Criterio	Pts	Descripción
Pseudocódigo: las tres fórmulas correctas	3 pts	1pt c/u
Python celsius_a_fahrenheit: fórmula y round()	4 pts	4/3/2/0
Python fahrenheit_a_celsius: fórmula y round()	3 pts	3/2/1/0
Python celsius_a_kelvin: suma 273.15 y round()	3 pts	3/2/1/0
Python: programa principal con las 3 temperaturas	2 pts	2/1/0

NIVEL AVANZADO — EJERCICIOS 6 Y 7**Ejercicio 6 Estadísticas de una lista****★★★ Avanzado****Puntaje: 15 pts****a) Pseudocódigo — encontrar_maximo**

```
FUNCION encontrar_maximo(lista)
    INICIO
        maximo <- lista[0]
        PARA CADA elemento EN lista HACER
            SI elemento > maximo ENTONCES
                maximo <- elemento
            FIN SI
        FIN PARA
        RETORNAR maximo
    FIN
FIN FUNCION
```

b) Pseudocódigo — mostrar_estadisticas

```
FUNCION mostrar_estadisticas(lista)
    INICIO
        ESCRIBIR "Lista: " + lista
        ESCRIBIR "Suma: " + calcular_suma(lista)
        ESCRIBIR "Promedio: " + calcular_promedio(lista)
        ESCRIBIR "Máximo: " + encontrar_maximo(lista)
        ESCRIBIR "Mínimo: " + encontrar_minimo(lista)
    FIN
FIN FUNCION
```

c) Código Python modelo

```
def calcular_suma(lista):
    total = 0
    for n in lista:
        total += n
    return total

def calcular_promedio(lista):
    return round(calcular_suma(lista) / len(lista), 2)

def encontrar_maximo(lista):
    maximo = lista[0]
    for n in lista:
        if n > maximo:
            maximo = n
    return maximo

def encontrar_minimo(lista):
    minimo = lista[0]
    for n in lista:
        if n < minimo:
            minimo = n
    return minimo

def mostrar_estadisticas(lista):
    print(f"Lista:      {lista}")
    print(f"Suma:        {calcular_suma(lista)}")
    print(f"Promedio:    {calcular_promedio(lista)}")
    print(f"Máximo:     {encontrar_maximo(lista)}")
    print(f"Mínimo:     {encontrar_minimo(lista)}")

numeros = [45, 82, 17, 63, 29, 91, 54, 38]
mostrar_estadisticas(numeros)
```

Rúbrica de corrección

Criterio	Pts	Descripción
Pseudocódigo encontrar_maximo: lógica del for/if correcta	2 pts	2/1/0
Pseudocódigo mostrar_estadisticas: llama las 4 funciones	1 pt	1/0
Python calcular_suma: acumulador con for	2 pts	2/1/0
Python calcular_promedio: usa calcular_suma()	2 pts	2/1/0
Python encontrar_maximo y encontrar_minimo sin max()/min()	4 pts	2pts c/u
Python mostrar_estadisticas: llama e imprime correctamente	4 pts	4/3/2/1/0



Si usan max()/min() en las funciones encontrar_: descontar 2pts de esas funciones.
 Aceptar que calcular_promedio use sum() directamente si calcular_suma no existe.
 Verificar que calcular_promedio llame a calcular_suma (composición de funciones).

Ejercicio 7 Validador de contraseña

★★★★ Avanzado

Puntaje: **15 pts****a) Pseudocódigo — tiene_mayuscula**

```
FUNCION tiene_mayuscula(clave)
  INICIO
    PARA CADA caracter EN clave HACER
      SI caracter ES mayúscula ENTONCES
        RETORNAR VERDADERO
      FIN SI
    FIN PARA
  RETORNAR FALSO
FIN
FIN FUNCION
```

b) Pseudocódigo — es_contrasena_valida

```
FUNCION es_contrasena_valida(clave)
  INICIO
    SI tiene_longitud_minima(clave, 8) Y
       tiene_mayuscula(clave) Y
       tiene_numero(clave) ENTONCES
      RETORNAR VERDADERO
    SINO
      RETORNAR FALSO
    FIN SI
  FIN
FIN FUNCION
```

c) Código Python modelo

```
def tiene_longitud_minima(clave, minimo=8):
    return len(clave) >= minimo

def tiene_mayuscula(clave):
    for c in clave:
        if c.isupper():
            return True
    return False

def tiene_numero(clave):
    for c in clave:
        if c.isdigit():
            return True
    return False

def es_contrasena_valida(clave):
    return (tiene_longitud_minima(clave) and
            tiene_mayuscula(clave) and
            tiene_numero(clave))

def evaluar_contrasena(clave):
    print(f"Contraseña: '{clave}'")
```



```

checks = [
    ("Longitud mínima (8)", tiene_longitud_minima(clave)),
    ("Contiene mayúscula", tiene_mayuscula(clave)),
    ("Contiene número", tiene_numero(clave)),
]
for nombre, ok in checks:
    simbolo = "✓" if ok else "X"
    estado = "SI" if ok else "NO"
    print(f" [{simbolo}] {nombre}: {estado}")
valida = "VÁLIDA" if es_contrasena_valida(clave) else "INVÁLIDA"
print(f" → Contraseña {valida}")

evaluar_contrasena("hola")
print()
evaluar_contrasena("Segura123")

```

Rúbrica de corrección

Criterio	Pts	Descripción
Pseudocódigo tiene_mayuscula: for + if + return	2 pts	2/1/0
Pseudocódigo es_contrasena_valida: lógica AND correcta	1 pt	1/0
Python tiene_longitud_minima con parámetro default	2 pts	2/1/0
Python tiene_mayuscula y tiene_numero (for + isupper/isdigit)	4 pts	2pts c/u
Python es_contrasena_valida llama las tres funciones	2 pts	2/1/0
Python evaluar_contrasena: muestra estado de cada condición	4 pts	4/3/2/1/0

EJERCICIO BONUS — EJERCICIO 8

Ejercicio 8 Mini sistema estudiantil

★★★ Avanzado

Puntaje: 10
(bonus) pts

Código Python modelo

```

# Diccionario global de estudiantes
estudiantes = {}

def agregar_estudiante(nombre, notas):
    estudiantes[nombre] = notas

def calcular_promedio_estudiante(nombre):
    notas = estudiantes[nombre]
    return round(sum(notas) / len(notas), 2)

def mejor_estudiante():
    mejor = None
    mejor_prom = -1

```

```

    for nombre in estudiantes:
        prom = calcular_promedio_estudiante(nombre)
        if prom > mejor_prom:
            mejor_prom = prom
            mejor = nombre
    return mejor

def mostrar_reporte():
    print("=" * 45)
    print(f"{'Nombre':<15} {'Promedio':>10} {'Estado':>15}")
    print("=" * 45)
    for nombre in estudiantes:
        p = calcular_promedio_estudiante(nombre)
        est = "Aprobado" if p >= 70 else ("Recuperación" if p >= 50 else
"Reprobado")
        print(f"{nombre:<15} {p:>10} {est:>15}")
    print("=" * 45)
    print(f"Mejor estudiante: {mejor_estudiante()}")

# Cargar datos
agregar_estudiante("Ana", [85, 90, 78])
agregar_estudiante("Luis", [60, 55, 70])
agregar_estudiante("María", [45, 40, 50])
agregar_estudiante("Carlos", [95, 88, 92])

mostrar_reporte()

```

Rúbrica de corrección

Criterio	Pts	Descripción
Diccionario global con 4+ estudiantes correctamente cargado	2 pts	2/1/0
agregar_estudiante y calcular_promedio_estudiante	2 pts	1pt c/u
mejor_estudiante: lógica de comparación correcta	2 pts	2/1/0
mostrar_reporte: itera, calcula promedio y estado de cada uno	4 pts	4/3/2/1/0